

Concurrent Database Access — Student 2 (Owners / Laslo)

Section 4.4 write-up for bungalow and boat owner conflict scenarios in FishyFinds.

Relevant code: `BungalowController`, `BoatController`, `TermController`, `TermService`, `ReservationService`, `Term` / `Reservation (@Version)`, image save during `addNewBungalow` / `addNewBoat`.

1. Owner creates reservation while client books the same slot

Situation

An owner defines or books a period on an offer at the same time a customer calls `makeReservation` for an overlapping interval on the same term.

Endpoints / classes

- Customer: `ReservationController.makeReservation` → `ReservationService.makeReservation`
- Owner: `TermController.addNewTermToOffer` → `TermService.addNewTermToOffer`
- Entities: `Term (@Version)`, `Reservation (@Version)`

Lock approach

Optimistic locking via `@Version` on `Term` and `Reservation`. The second commit that updates the same term version fails.

Why chosen

Owner and client paths converge on the same term entity. Optimistic locking fits interactive UI flows where conflicts are uncommon.

2. Owner defines an action while client reserves

Situation

The owner creates a quick action (`ReservationType.QUICK`) while a customer books an overlapping default reservation or claims another action.

Endpoints / classes

- Owner: `POST /api/createQuickAction` → `ReservationController.createQuickAction` → `ReservationService.createQuickAction` (`ReservationType.QUICK` ; overlap check via `hasOverlappingActiveReservation`)
- Also: `ReservationService.getActionsForOffer`
- Customer: `makeReservation` / `makeReservationAction`
- Entity: `Reservation` with `@Version`

Note on `createQuickAction`

`createQuickAction` is the concrete owner write path that races with client `makeReservation` : both can pass an overlap check on a stale snapshot and insert conflicting active rows. Optimistic `@Version` on `Reservation` / `Term` (and/or tightening the overlap check inside the same transaction) makes the losing writer fail instead of silently double-booking.

Lock approach

Optimistic locking on `Reservation` (and `Term` when linked into the term's reservation list).

Why chosen

Actions are stored as reservation rows; versioning those rows matches the customer-side action race solution.

3. Delete offer while it is reserved

Situation

An owner deletes a bungalow/boat offer that still has active upcoming reservations, racing with new bookings.

Endpoints / classes

- `BungalowController` / `BoatController` / `OfferService`
- `ReservationService`

Lock approach

Business guard (reject delete if active reservations exist) plus **optimistic locking** on related `Reservation / Term` rows.

Why chosen

Deleting an offer is rare and high-impact; explicit reservation checks are clearer than locks alone.

Extra: concurrent addNewBungalow image save / two owners editing same offer

Situation

Parallel `POST /api/addNewBungalow` image saves can overwrite files named from offer count. Two sessions editing the same offer can last-write-wins on metadata.

Endpoints / classes

- `BungalowController.addNewBungalow` → `BungalowService.addNewBungalow`
- `BoatController` / `BoatService.addNewBoat` , `ImageItem` , `ImageService`

Lock approach

Add `@Version` on `Offer` for metadata races; use unique image filenames (UUID) or serialize `saveImage` per offer.

Why chosen

File-system writes are not covered by JPA `@Version` alone.

Summary

Concern	Mechanism
Term availability	<code>@Version</code> on <code>Term</code>
Reservation / action	<code>@Version</code> on <code>Reservation</code>

Penalty vs book	Pessimistic lock on <code>CustomerRepository.findByEmail</code>
Images	Unique paths / short critical section